

Namaste Python

Visual Revision Guide — Version 2

More code examples + chapter-wise interview questions



NAMASTE PYTHON

Concept-First Revision Guide

Understand • Remember • Apply • Interview Ready

Python in One Line

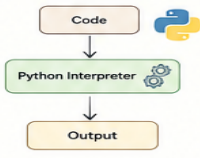
Python is a high-level, interpreted, general-purpose programming language with simple syntax and powerful libraries.

ROADMAP

- Python Basics
- Data Types & Variables
- Operators
- Control Flow
- Functions
- *args & **kwargs
- Scope & LEGB Rule
- Closures & Decorators
- Iterators & Generators
- OOP (Object Oriented Programming)
- Exceptions
- Context Managers
- Modules & Packages
- Python Internals
- Async Python
- FastAPI Basics
- Pydantic
- SQLAlchemy Basics
- Testing with pytest
- Logging
- Interview Q & A
- Rapid Revision Sheet

1. PYTHON BASICS

- Python code is executed line by line.
- Indentation (spaces) is mandatory.
- Everything is an object in Python.



2. VARIABLES & DATA TYPES

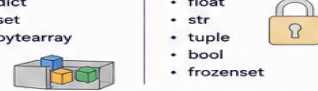
```
x = 10 # int
pi = 3.14 # float
name = "Alice" # str
is_valid = True # bool
items = [1, 2, 3] # list
person = {"name": "Bob"} # dict
unique = {1, 2, 3} # set
point = (1, 2) # tuple
```

int	float	str	bool
10	3.14	"Hi"	True

list	tuple	set	dict
[1,2,3]	(1,2)	{1,2,3}	{'a':1}

3. MUTABLE vs IMMUTABLE

Mutable (Can be changed)	Immutable (Cannot be changed)
- list - dict - set - bytearray	- int - float - str - tuple - bool - frozenset



4. == vs is

== (Value comparison)
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b) # True

is (Identity comparison)
a = [1, 2, 3]
b = a
print(a is b) # True

Use == to compare values
Use is to check same object in memory

5. CONTROL FLOW

```
if / elif / else
x = 10
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")

for loop
for i in range(3):
    print(i) # 0 1 2

while loop
i = 0
while i < 3:
    print(i)
    i += 1
```

6. FUNCTIONS

```
def greet(name):
    """This is a docstring"""
    return f"Hello, {name}!"

print(greet("Alice"))
```

Types of Functions

- Built-in (len(), print(), type())
- User-defined
- Lambda (anonymous)

Lambda Example

```
square = lambda x: x ** 2
print(square(4)) # 16
```

7. *args & **kwargs

***args (Non-keyword arguments)**

```
def add(*numbers):
    return sum(numbers)

print(add(1, 2, 3, 4)) # 10
```

****kwargs (Keyword arguments)**

```
def info(**data):
    for k, v in data.items():
        print(k, "=", v)

info(name="Alice", age=25)
```

WHY PYTHON?

- Easy to learn
- Readable & clean syntax
- Huge standard library
- Cross-platform
- Great for Web, Data, AI/ML, Automation & more

TIP

Focus more on concepts than syntax. Understand 'why' something works, not just 'how'.



8. SCOPE & LEGB RULE

Python looks for a variable in this order:

- L** Local (inside current function)
- E** Enclosing (inside enclosing function)
- G** Global (in global space)
- B** Built-in (python built-in names)

If not found at any level, NameError is raised.

9. CLOSURES & DECORATORS

Closure Example

```
def outer(msg):
    def inner():
        print(msg)
    return inner

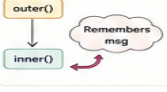
hello = outer("Hi")
hello() # Hi
```

Decorator Example

```
def decor(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper

@decor
def say_hello():
    print("Hello")

say_hello()
```



10. ITERATORS & GENERATORS

Iterator

```
it = iter([1,2,3])
print(next(it)) # 1
print(next(it)) # 2
print(next(it)) # 3
```

Generator (using yield)

```
def count(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for num in count(3):
    print(num)
```

Generators are memory efficient (lazy evaluation).

11. OOP IN PYTHON

Class & Object

```
class Person:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"Hi, I am {self.name}"

p = Person("Alice")
print(p.greet())
```

Pillars of OOP

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

12. EXCEPTIONS

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("Cannot divide by zero")
else:
    print("No Exception")
finally:
    print("This will always run")
```

Common Exceptions

- ZeroDivisionError
- ValueError
- TypeError
- FileNotFoundError
- KeyError

13. CONTEXT MANAGERS

```
with open("file.txt", "r") as f:
    data = f.read()
# file is automatically closed
```

Custom Context Manager

```
class MyContext:
    def __enter__(self):
        print("Enter")
    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Exit")

with MyContext():
    print("Inside block")
```

14. PYTHON MEMORY MODEL (HIGH LEVEL)

Stack (Primitive)
int, float, bool

Heap (Objects)
list, dict, class, etc.

Objects in heap are referenced by variables in stack.

15. GIL (GLOBAL INTERPRETER LOCK)

- Allows only one thread to execute Python bytecode at a time.
- Good for I/O-bound tasks (not CPU-bound).
- Multiprocessing is used for CPU-bound tasks.

16. ASYNC IN PYTHON

```
import asyncio
async def main():
    print("Start")
    await asyncio.sleep(1)
    print("End")

asyncio.run(main())
```



★ **REMEMBER:** Practice code daily • Build projects • Solve problems • Revise regularly → You will master Python! 🚀

Chapter 1 — Python Fundamentals

Understand objects, variables, types, mutability, operators, and collections.

Variables & objects

Simple: A variable is a name that refers to an object.

Mental model: name → object in memory → value/type

Example 1 — rebinding

```
x = 10
y = x
x = 20
print(x) # 20
print(y) # 10
```

Changing x later does not change y because x is rebound.

Example 2 — shared mutable object

```
a = [1, 2]
b = a
b.append(3)
print(a) # [1, 2, 3]
```

Both names refer to the same list.

Remember: Think in terms of names and objects, not boxes containing values.

Core data types

Simple: Python provides built-in types for numbers, text, booleans, sequences, sets, and mappings.

Mental model: int / float / bool / str / list / tuple / set / dict

Example

```
age = 30
price = 99.5
active = True
name = 'Sam'
items = [1, 2]
point = (10, 20)
tags = {'python', 'api'}
user = {'name': 'Sam'}
```

Know which types are mutable and which are immutable.

Operators

Simple: Operators perform arithmetic, comparison, logical, membership, and identity operations.

Example

```
a = 10
b = 3
print(a + b) # 13
print(a // b) # 3
print(a % b) # 1
print(a > b) # True
print(2 in [1, 2, 3]) # True
```

Use `//` for floor division and `%` for remainder.

Collections

Simple: Lists preserve order and are mutable; tuples are immutable; sets contain unique elements; dictionaries map keys to values.

List

```
items = [10, 20, 30]
items.append(40)
items.pop()
print(items)
```

Dictionary

```
user = {'name': 'Sam', 'age': 30}
print(user['name'])
user['city'] = 'Kolkata'
```

Set

```
tags = {'python', 'api', 'python'}
print(tags) # duplicates removed
```

Chapter Interview Questions

1. What is the difference between a variable and an object?

Expected answer: A variable is a name bound to an object; the object contains the actual value and type.

2. What are mutable and immutable objects?

Expected answer: Mutable objects can change in place; immutable objects cannot be changed in place.

3. List vs tuple?

Expected answer: Both are ordered sequences, but lists are mutable and tuples are immutable.

4. List vs set?

Expected answer: Lists preserve duplicates and order; sets store unique elements and are optimized for membership.

5. Why are dictionaries fast for lookup?

Expected answer: They use a hash table internally, giving average O(1) key lookup.

6. What is the difference between `==` and `is`?

Expected answer: `==` compares values; `is` compares object identity.

Chapter 2 — Control Flow & Pythonic Iteration

if / elif / else

Simple: Conditional statements choose which block executes based on a condition.

Example

```
score = 78
if score >= 90:
    grade = 'A'
elif score >= 60:
    grade = 'B'
else:
    grade = 'C'
print(grade)
```

for loop

Simple: A for loop iterates over an iterable.

Example

```
for name in ['A', 'B', 'C']:
    print(name)
```

enumerate

```
names = ['A', 'B']
for index, name in enumerate(names):
    print(index, name)
```

Prefer enumerate over range(len(...)) when you need both index and value.

while loop

Simple: A while loop continues while its condition remains true.

Example

```
i = 0
while i < 3:
    print(i)
    i += 1
```

Always make sure the loop can eventually terminate.

zip and unpacking

Simple: zip combines iterables and unpacking assigns elements to multiple names.

Example

```
names = ['A', 'B']
ages = [20, 30]
for name, age in zip(names, ages):
    print(name, age)
```

Unpacking

```
first, *middle, last = [1, 2, 3, 4]
print(first) # 1
print(middle) # [2, 3]
print(last) # 4
```

Comprehensions

Simple: Comprehensions build collections concisely.

List comprehension

```
squares = [x * x for x in range(6)]
```

Dict comprehension

```
lookup = {x: x*x for x in range(4)}
```

Set comprehension

```
unique_lengths = {len(x) for x in ['a', 'bb', 'a']}
```

Chapter Interview Questions

1. Why is `enumerate()` preferred over `range(len(items))`?

Expected answer: It directly expresses that you need both the index and value and is generally clearer.

2. What does `zip()` do with unequal iterables?

Expected answer: Normal zip stops when the shortest iterable is exhausted.

3. What is a list comprehension?

Expected answer: A concise syntax for creating a list from an iterable, optionally with filtering.

4. When should you avoid comprehensions?

Expected answer: When they become deeply nested or reduce readability.

5. What is the difference between `break` and `continue`?

Expected answer: `break` exits the loop; `continue` skips the current iteration and proceeds to the next.

Chapter 3 — Functions, *args, **kwargs & Scope

Function basics

Simple: Functions package reusable behavior and can return values.

Example

```
def add(a, b):  
    return a + b  
  
result = add(2, 3)  
print(result) # 5
```

Default and keyword arguments

Simple: Arguments can be passed positionally or by name.

Example

```
def greet(name, message='Hello'):  
    return f'{message}, {name}'  
  
print(greet('Sam'))  
print(greet('Sam', message='Hi'))
```

*args

Simple: `\*args` collects extra positional arguments into a tuple.

Example

```
def total(*numbers):  
    return sum(numbers)  
  
print(total(1, 2, 3, 4)) # 10
```

**kwargs

Simple: `\*\*kwargs` collects extra keyword arguments into a dictionary.

Example

```
def show_user(**data):  
    for key, value in data.items():  
        print(key, value)  
  
show_user(name='Sam', age=30)
```

LEGB scope

Simple: Python searches Local, Enclosing, Global, then Built-in scopes.

Example

```
x = 'global'  
  
def outer():  
    x = 'enclosing'  
    def inner():  
        print(x)  
    inner()  
  
outer() # enclosing
```

global and nonlocal

Simple: global changes a global binding; nonlocal changes a binding in an enclosing function.

Example

```
count = 0  
  
def increment():  
    global count  
    count += 1
```

```
increment()  
print(count) # 1
```

nonlocal

```
def counter():  
    value = 0  
    def inc():  
        nonlocal value  
        value += 1  
    return value  
    return inc  
  
next_count = counter()  
print(next_count()) # 1  
print(next_count()) # 2
```

Chapter Interview Questions

1. What are *args and **kwargs?

Expected answer: *args collects extra positional arguments; **kwargs collects extra keyword arguments.

2. What is LEGB?

Expected answer: Local, Enclosing, Global, Built-in: Python's name resolution order.

3. What is the difference between global and nonlocal?

Expected answer: global targets module-level scope; nonlocal targets a variable in an enclosing function scope.

4. Are functions first-class objects in Python?

Expected answer: Yes. They can be assigned, passed, returned, and stored in collections.

5. What is a higher-order function?

Expected answer: A function that accepts another function or returns a function.

Chapter 4 — Lambda, Closures & Decorators

Lambda

Simple: A lambda is a small anonymous function.

Example

```
square = lambda x: x * x
print(square(5)) # 25

items = [3, 1, 2]
print(sorted(items, key=lambda x: x))
```

Use lambdas for short expressions; use def when the logic deserves a name.

Closure

Simple: An inner function can remember variables from its enclosing function.

Example

```
def multiplier(n):
    def multiply(x):
        return x * n
    return multiply

double = multiplier(2)
print(double(10)) # 20
```

Decorator

Simple: A decorator wraps a callable to add behavior before/after it.

Example

```
from functools import wraps

def log_call(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        print('Before')
        result = fn(*args, **kwargs)
        print('After')
        return result
    return wrapper

@log_call
def greet(name):
    return f'Hello {name}'

print(greet('Sam'))
```

The decorator syntax `@log_call` is essentially `greet = log_call(greet)`.

Chapter Interview Questions

1. What is a closure?

Expected answer: A function that retains access to variables from an enclosing scope after the outer function has returned.

2. What is a decorator?

Expected answer: A callable that receives another callable and returns a wrapped/modified callable.

3. Why use `functools.wraps`?

Expected answer: It preserves metadata such as the wrapped function's name and docstring.

4. What does `@decorator` mean?

Expected answer: It is syntactic sugar for assigning the decorated function to `decorator(function)`.

5. Decorator vs middleware?

Expected answer: Both wrap behavior, but middleware commonly surrounds a broader request/response pipeline while decorators usually wrap a callable.

Chapter 5 — Iterators & Generators

Iterable vs iterator

Simple: An iterable can produce an iterator; an iterator keeps state and returns the next item.

Example

```
items = [10, 20, 30]
it = iter(items)
print(next(it)) # 10
print(next(it)) # 20
```

Generator function

Simple: A function containing yield becomes a generator function.

Example

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for n in count_up_to(3):
    print(n)
```

Execution pauses at each yield and resumes when the next value is requested.

Generator expression

Simple: Generator expressions are lazy versions of many list comprehensions.

Example

```
numbers = (x*x for x in range(1_000_000))
print(next(numbers))
```

The million results are not created in memory at once.

yield from

Simple: yield from delegates iteration to another iterable or generator.

Example

```
def numbers():
    yield from [1, 2, 3]
    yield from [4, 5]

print(list(numbers())) # [1, 2, 3, 4, 5]
```

Chapter Interview Questions

1. Iterable vs iterator?

Expected answer: An iterable can produce an iterator; an iterator implements iteration state and next().

2. Why are generators memory efficient?

Expected answer: They yield one value at a time instead of materializing the complete sequence.

3. What happens when yield is executed?

Expected answer: The generator pauses and returns a value; its execution state is retained for the next request.

4. Generator vs list comprehension?

Expected answer: A list comprehension creates the whole list immediately; a generator expression evaluates lazily.

5. What is StopIteration?

Expected answer: The signal used by an iterator to indicate that no more values are available.

Chapter 6 — OOP, Classmethod, Staticmethod, Property & MRO

Class and object

Simple: A class defines behavior and structure; an object is an instance.

Example

```
class User:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f'Hello {self.name}'

u = User('Sam')
print(u.greet())
```

classmethod

Simple: A class method receives cls and can work with class-level state.

Example

```
class User:
    count = 0

    @classmethod
    def total(cls):
        return cls.count

print(User.total())
```

staticmethod

Simple: A static method is namespaced inside the class but receives no automatic self/cls.

Example

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b

print(Math.add(2, 3))
```

property

Simple: A property exposes method logic through attribute-style access.

Example

```
class User:
    def __init__(self, age):
        self._age = age

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError('Invalid age')
        self._age = value
```

Inheritance and super

Simple: Inheritance lets a class extend another class; super() follows the MRO.

Example

```
class Animal:
    def speak(self):
```

```
return 'sound'

class Dog(Animal):
    def speak(self):
        return super().speak() + ' bark'

print(Dog().speak())
```

Chapter Interview Questions

1. self vs cls?

Expected answer: self refers to an instance; cls refers to the class in a classmethod.

2. classmethod vs staticmethod?

Expected answer: classmethod receives cls and can access class state; staticmethod receives neither automatically.

3. What is @property used for?

Expected answer: To expose controlled attribute-style access while keeping method logic.

4. What is MRO?

Expected answer: Method Resolution Order is the order Python follows when searching classes for attributes/methods.

5. What does super() do?

Expected answer: It delegates lookup to the next class in the MRO rather than simply meaning 'parent class'.

6. What is polymorphism?

Expected answer: Different objects can implement the same interface and be used through that common interface.

Chapter 7 — Exceptions, Files & Context Managers

try / except / else / finally

Simple: Use try for risky code, except for handling expected failures, else when no exception occurred, and finally for cleanup.

Example

```
try:
    value = int('10')
except ValueError:
    print('Bad input')
else:
    print('Success:', value)
finally:
    print('Done')
```

Raise your own exception

Simple: Use raise when a function cannot continue with invalid input or state.

Example

```
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError('Insufficient balance')
    return balance - amount
```

File handling

Simple: with ensures the file is closed even if an error occurs.

Example

```
with open('notes.txt', 'w') as f:
    f.write('Python revision')

with open('notes.txt', 'r') as f:
    text = f.read()
```

Custom context manager

Simple: A class can implement `__enter__` and `__exit__` to become a context manager.

Example

```
class MyContext:
    def __enter__(self):
        print('Enter')
        return self

    def __exit__(self, exc_type, exc, tb):
        print('Exit')

with MyContext():
    print('Inside')
```

Chapter Interview Questions

1. Why should you catch specific exceptions?

Expected answer: It avoids hiding unrelated bugs and makes error handling intentional.

2. What is finally used for?

Expected answer: Cleanup that should happen whether or not an exception occurred.

3. What is the purpose of raise?

Expected answer: To explicitly signal an error or propagate a condition to the caller.

4. Why use with open(...)?

Expected answer: It guarantees resource cleanup, including closing the file.

5. What are `__enter__` and `__exit__`?

Expected answer: They define setup and cleanup behavior for a context manager.

Chapter 8 — Copying, Memory Management & GIL

Shallow copy

Simple: Copies the outer container but keeps references to nested objects.

Example

```
import copy

a = [[1], [2]]
b = copy.copy(a)
b[0].append(99)
print(a) # [[1, 99], [2]]
```

Deep copy

Simple: Recursively copies nested objects.

Example

```
import copy

a = [[1], [2]]
b = copy.deepcopy(a)
b[0].append(99)
print(a) # [[1], [2]]
```

Reference counting

Simple: CPython tracks references to objects and can reclaim objects whose reference count reaches zero.

Example

```
a = []
b = a
print(a is b) # True

del a
print(b) # object still exists
```

GIL

Simple: The standard CPython GIL historically allows only one thread at a time to execute Python bytecode within an interpreter.

Concept

```
# I/O-bound: threads/async can be useful
# CPU-bound: multiprocessing is often useful
```

Choose the concurrency model based on the workload.

Chapter Interview Questions

1. Shallow vs deep copy?

Expected answer: Shallow copies the outer object but shares nested references; deep copy recursively copies nested objects.

2. What does del do?

Expected answer: It removes a name/reference; it does not necessarily immediately destroy the object.

3. How does CPython manage memory?

Expected answer: Primarily reference counting plus cyclic garbage collection.

4. What is the GIL?

Expected answer: A CPython interpreter lock that historically prevents multiple threads from simultaneously executing Python bytecode in the same interpreter.

5. Threading vs multiprocessing?

Expected answer: Threads are often useful for I/O-bound workloads; multiprocessing can provide parallel CPU execution.

Chapter 9 — Async Python, asyncio & Event Loop

async / await

Simple: `async def` creates a coroutine function; `await` pauses the current coroutine until an awaitable can continue.

Example

```
import asyncio

async def fetch():
    await asyncio.sleep(1)
    return 'done'

result = asyncio.run(fetch())
print(result)
```

gather

Simple: `gather` runs multiple awaitables concurrently from the perspective of the event loop and returns their results.

Example

```
async def fetch(name):
    await asyncio.sleep(1)
    return name

async def main():
    results = await asyncio.gather(
        fetch('A'), fetch('B')
    )
    print(results)
```

Two one-second waits can complete in roughly the same I/O waiting window rather than sequentially.

create_task

Simple: `create_task` schedules a coroutine as a `Task` so it can progress while other async work runs.

Example

```
async def main():
    task = asyncio.create_task(fetch('A'))
    result = await task
    print(result)
```

Blocking code in async

Simple: A blocking call can freeze the event loop and delay every other task.

Bad

```
async def endpoint():
    time.sleep(5) # blocks event loop
```

Do not use blocking calls in async code unless deliberately moved to a worker/thread.

Better

```
async def endpoint():
    await asyncio.sleep(5)
```

Use an async API when the underlying operation is asynchronous.

Chapter Interview Questions

1. What is a coroutine?

Expected answer: An awaitable object representing suspended/resumable asynchronous computation.

2. What does `await` do?

Expected answer: It suspends the current coroutine until the awaited operation can provide a result, allowing other tasks to run.

3. What is the event loop?

Expected answer: The scheduler that drives asynchronous tasks and resumes them when I/O or other awaitables are ready.

4. `gather` vs `create_task`?

Expected answer: `create_task` schedules a coroutine as a task; `gather` waits for multiple awaitables and collects their results.

5. Why is blocking code dangerous in `async`?

Expected answer: It blocks the event loop, preventing other `async` tasks from making progress.

Chapter 10 — FastAPI

Basic endpoint

Simple: FastAPI maps HTTP routes to Python functions and automatically generates API documentation.

Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get('/hello')
def hello():
    return {'message': 'Hello'}
```

Pydantic request model

Simple: Pydantic validates structured request data before your business logic receives it.

Example

```
from pydantic import BaseModel

class UserCreate(BaseModel):
    name: str
    age: int

@app.post('/users')
def create_user(user: UserCreate):
    return user
```

Dependency injection

Simple: Depends lets FastAPI resolve reusable dependencies such as authentication or DB sessions.

Example

```
from fastapi import Depends

def get_current_user():
    return {'id': 123}

@app.get('/me')
def me(user=Depends(get_current_user)):
    return user
```

Response model

Simple: Response models define and validate the shape of outgoing API data.

Example

```
class UserOut(BaseModel):
    id: int
    name: str

@app.get('/users/1', response_model=UserOut)
def get_user():
    return {'id': 1, 'name': 'Sam'}
```

Async FastAPI endpoint

Simple: Use async when the operations you await are genuinely asynchronous.

Example

```
@app.get('/items')
async def get_items():
    rows = await fetch_items()
    return rows
```

Chapter Interview Questions

1. What is dependency injection in FastAPI?

Expected answer: A mechanism for declaring reusable dependencies that FastAPI resolves for each request.

2. Why use Pydantic models?

Expected answer: For input validation, parsing, serialization, and explicit API contracts.

3. When should an endpoint be async?

Expected answer: When it performs asynchronous I/O or awaits async operations.

4. What is response_model?

Expected answer: It defines the expected output schema and helps validate/serialize the response.

5. How would you get the current authenticated user?

Expected answer: Typically through a reusable dependency such as `Depends(get_current_user)`.

Chapter 11 — SQLAlchemy

Create and commit

Simple: The Session tracks ORM objects and manages their database transaction.

Example

```
user = User(name='Sam')
session.add(user)
session.commit()
session.refresh(user)
print(user.id)
```

flush vs commit

Simple: flush sends pending SQL to the database inside the current transaction; commit finalizes it.

Example

```
session.add(user)
session.flush()
print(user.id) # may now be populated
session.commit()
```

select

Simple: Modern SQLAlchemy uses SQL expressions with Session.execute/Session.scalars.

Example

```
from sqlalchemy import select

stmt = select(User).where(User.name == 'Sam')
user = session.scalar(stmt)
```

Relationship loading

Simple: Loading strategy controls when and how related rows are fetched.

selectinload

```
from sqlalchemy.orm import selectinload

stmt = select(User).options(selectinload(User.orders))
users = session.scalars(stmt).all()
```

Select-in loading can avoid the N+1 pattern for collections.

Transaction rollback

Simple: Rollback returns the current transaction to a clean state after a failed database operation.

Example

```
try:
    session.add(user)
    session.commit()
except Exception:
    session.rollback()
    raise
```

Chapter Interview Questions

1. What is the SQLAlchemy Session?

Expected answer: It manages ORM identity, pending changes, database communication, and transaction boundaries.

2. flush vs commit?

Expected answer: flush sends SQL within the current transaction; commit finalizes the transaction.

3. What is N+1?

Expected answer: A parent query followed by one additional query per parent for related data.

4. joinedload vs selectinload?

Expected answer: joinedload fetches related data via joins; selectinload generally uses a separate IN query for related collections.

5. Why rollback after an exception?

Expected answer: A failed transaction leaves the Session in a failed transactional state that must be rolled back before continuing.

Chapter 12 — Testing, Logging & Production Habits

pytest basics

Simple: Tests verify behavior and protect against regressions.

Example

```
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5
```

Fixtures

Simple: Fixtures provide reusable setup/teardown for tests.

Example

```
import pytest

@pytest.fixture
def user():
    return {'id': 1, 'name': 'Sam'}

def test_user(user):
    assert user['id'] == 1
```

Mocking

Simple: Mocking replaces an external dependency so a test can focus on one unit.

Example

```
from unittest.mock import Mock

api = Mock()
api.get.return_value = {'ok': True}

assert api.get()['ok'] is True
```

Logging

Simple: Logging records structured information about application behavior.

Example

```
import logging

logger = logging.getLogger(__name__)
logger.info('processing started')
logger.error('processing failed')
```

Use log levels appropriately and never log secrets or tokens.

Configuration

Simple: Configuration should come from environment/configuration rather than hard-coded secrets.

Example

```
import os

DATABASE_URL = os.getenv('DATABASE_URL')
DEBUG = os.getenv('DEBUG', 'false').lower() == 'true'
```

Chapter Interview Questions

1. What makes a good unit test?

Expected answer: It is focused, deterministic, readable, and tests observable behavior.

2. What is a pytest fixture?

Expected answer: Reusable test setup/teardown provided as a dependency to tests.

3. Why mock external services?

Expected answer: To isolate the unit under test, avoid network dependency, and make tests fast and deterministic.

4. Why use logging instead of print?

Expected answer: Logging provides levels, configuration, formatting, handlers, and production-friendly diagnostics.

5. Where should secrets live?

Expected answer: In secure environment/configuration management, not source code or Git.

Chapter 13 — Advanced Python Interview Rapid Fire

1. What is duck typing?

Expected answer: Python cares about supported behavior/protocols rather than an object's declared type.

2. What is EAFP?

Expected answer: Easier to Ask Forgiveness than Permission: try the operation and handle the expected exception.

3. What is LBYL?

Expected answer: Look Before You Leap: check conditions before performing an operation.

4. What is `__slots__`?

Expected answer: A mechanism that can restrict instance attributes and reduce per-instance memory overhead in suitable cases.

5. What is a dataclass?

Expected answer: A decorator-driven way to generate common methods for data-holding classes.

6. What is a protocol?

Expected answer: A structural typing interface describing behavior a type supports.

7. What is monkey patching?

Expected answer: Changing/replacing attributes or behavior at runtime.

8. What is dependency inversion?

Expected answer: High-level code should depend on abstractions rather than concrete low-level implementations.

9. What is `id()`?

Expected answer: It returns an object's identity value for the lifetime of that object.

10. What is hashability?

Expected answer: An object is hashable when it has a stable hash and can be used as a dictionary key or set member.

Final Revision Strategy

Use this 4-step loop for every chapter.

Step	What to do
1. Look	Study the visual/mental model for 2–3 minutes.
2. Explain	Close the PDF and explain the concept aloud in your own words.
3. Code	Write one small example without copying.
4. Interview	Answer the chapter questions without looking at the expected answer.

Recommended interview order: Python fundamentals → functions/scope → closures/decorators → iterators/generators → OOP → exceptions → memory/GIL → async → FastAPI → SQLAlchemy → testing.